

ITA0612-Machine Learning

CO3 - AT1 EXPERIMENTS

Divyesh S P (192424147)

Experiment 1: Naïve Bayes Classifier on Iris Dataset

Import Required Libraries

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

Load the Iris Dataset

Code

```
iris = load_iris()

X = iris.data
y = iris.target
```

Split the Dataset into Training and Testing Sets

Code

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Train the Gaussian Naïve Bayes Model

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

▼ GaussianNB ⓘ ?

```
GaussianNB()
```

Predict the Test Data

```
y_pred = model.predict(X_test)
```

```
print(y_pred)
```

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]
```

Calculate Accuracy

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9777777777777777
```

Calculate Precision

Code

```
from sklearn.metrics import precision_score

precision = precision_score(y_test, y_pred, average='weighted')
print("Precision:", precision)
```

Precision: 0.9793650793650793

Calculate Recall

```
from sklearn.metrics import recall_score

recall = recall_score(y_test, y_pred, average='weighted')
print("Recall:", recall)
```

Recall: 0.9777777777777777

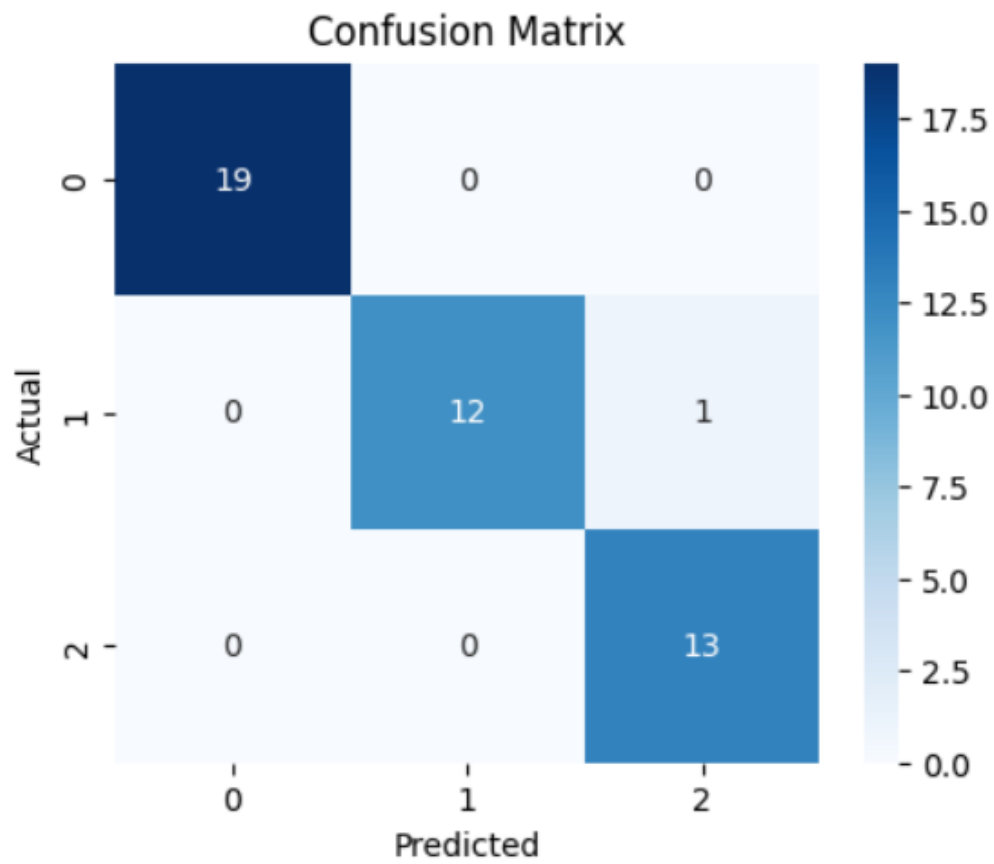
Classification Report

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.92	0.96	13
2	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Confusion Matrix (All Features)



Experiment Using Sepal Features

Train Model Using Sepal Features

```
#only sepal features
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()

X = iris.data[:, :2]      # Sepal Length and Sepal Width
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

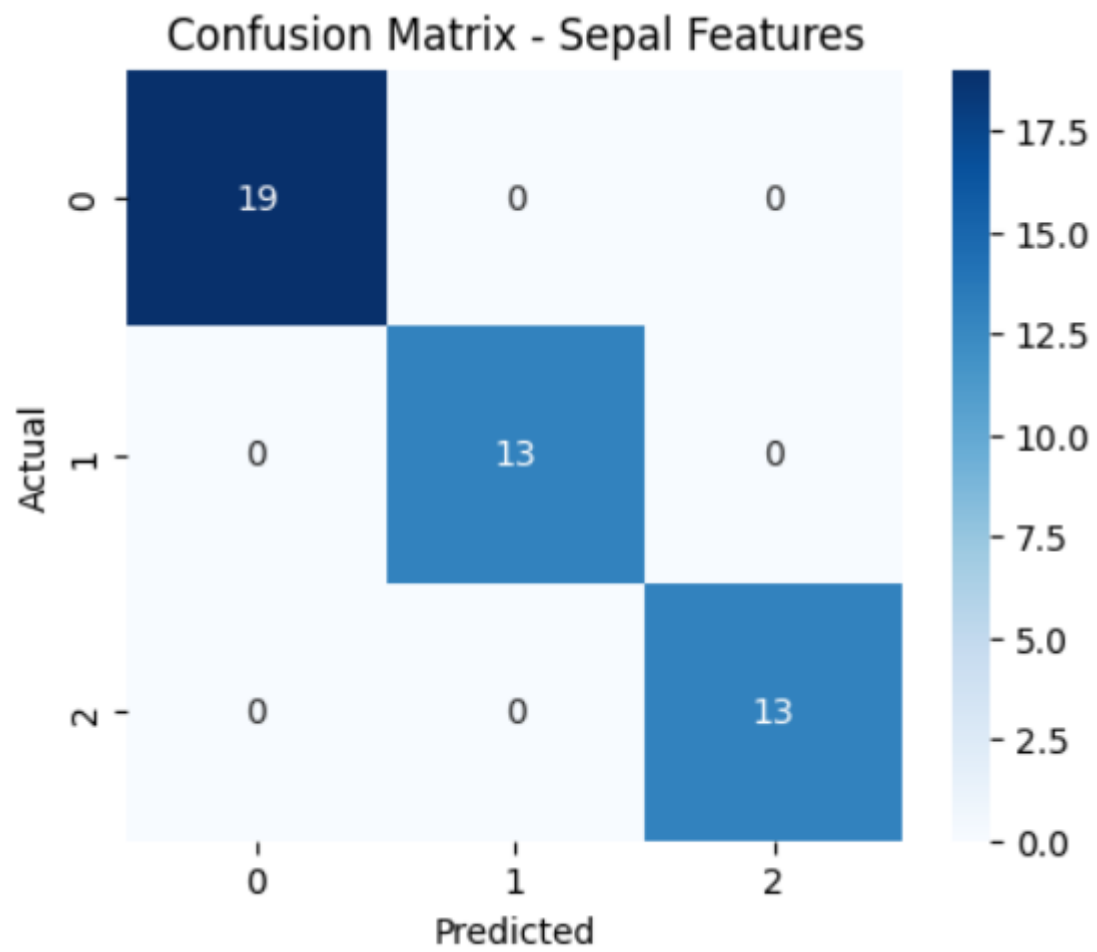
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Sepal Features")
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall   :", recall_score(y_test, y_pred, average='weighted'))
```

```
Sepal Features
Accuracy : 0.8222222222222222
Precision: 0.833841684822077
Recall   : 0.8222222222222222
```

Confusion Matrix (Sepal Features)



Experiment Using Petal Features

Train Model Using Petal Features

```
#only petal features
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score

iris = load_iris()

X = iris.data[:, 2:]      # Petal Length and Petal Width
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

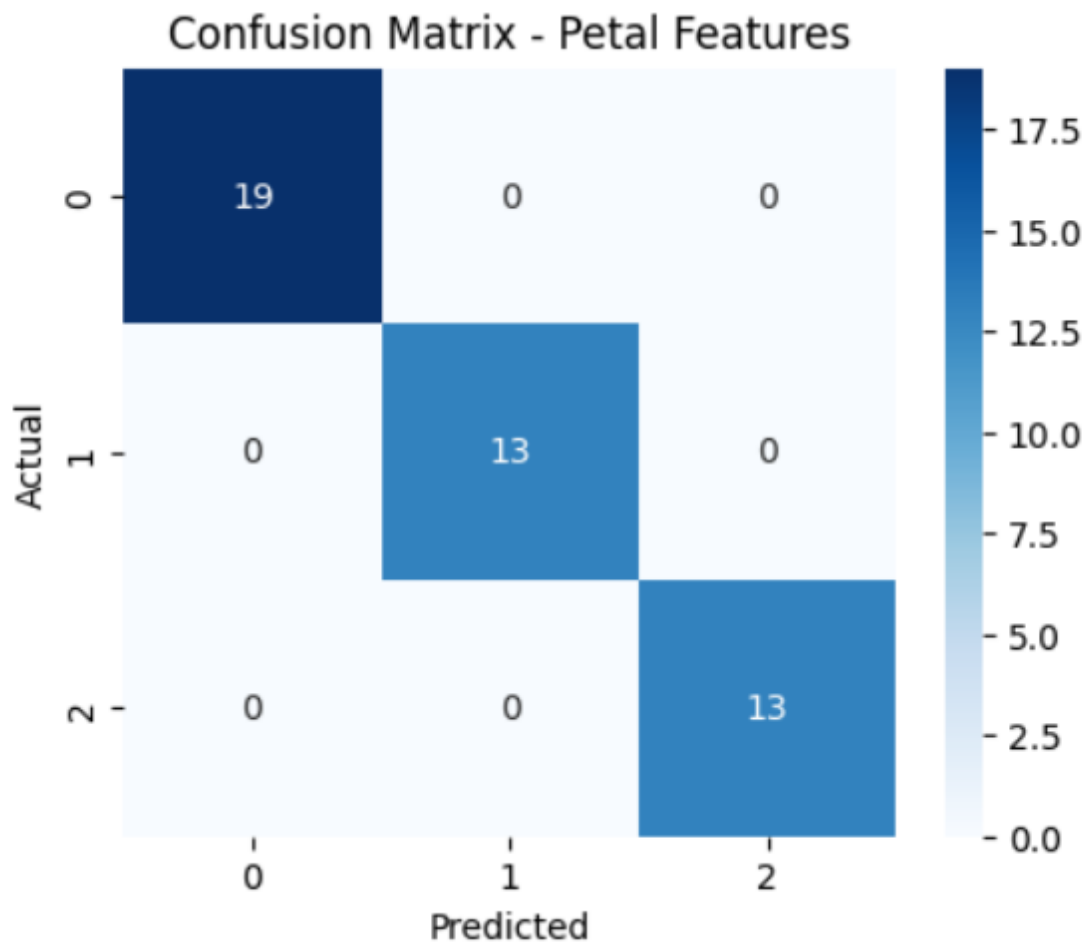
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Petal Features")
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall   :", recall_score(y_test, y_pred, average='weighted'))
```

Petal Features
Accuracy : 1.0
Precision: 1.0
Recall : 1.0

Confusion Matrix (Petal Features)



Experiment Using Smoothing

Compare Different Smoothing Values

```
#compare smoothing
model = GaussianNB(var_smoothing=1e-7)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print("Recall   :", recall_score(y_test, y_pred, average='weighted'))
```

Accuracy : 1.0
Precision: 1.0
Recall : 1.0

Performance Comparison Table

	Feature Set	Accuracy	Precision	Recall
0	All Features	0.9778	0.9788	0.9778
1	Sepal Features	0.8222	0.8338	0.8222
2	Petal Features	1.0000	1.0000	1.0000
3	Smoothing (1e-7)	1.0000	1.0000	1.0000

Experiment 2: MLE vs Bayesian Estimation

Load Iris Dataset

```
feature = iris.data[:,2]
```

```
small_sample = feature[:20]
```

```
large_sample = feature
```

```
prior_mean = 4  
prior_weight = 10
```

Small Sample (20 Samples)

```
import numpy as np  
from sklearn.datasets import load_iris  
  
# Load dataset  
iris = load_iris()  
  
# Select Petal Length feature  
feature = iris.data[:, 2]  
  
# Small sample (20 samples)  
small_sample = feature[:20]  
  
# MLE Mean  
mle_mean = np.mean(small_sample)  
  
# MLE Variance  
mle_variance = np.var(small_sample)  
  
# Bayesian Estimation  
prior_mean = 4  
prior_weight = 10  
  
bayesian_mean = (  
    prior_weight * prior_mean +  
    len(small_sample) * mle_mean  
) / (prior_weight + len(small_sample))  
  
print("Small Sample (20 Samples)")  
print("MLE Mean:", mle_mean)  
print("MLE Variance:", mle_variance)  
print("Bayesian Mean:", bayesian_mean)
```

```
Small Sample (20 Samples)
MLE Mean: 1.4349999999999998
MLE Variance: 0.020274999999999998
Bayesian Mean: 2.2899999999999996
```

Large Sample (150 Samples)

```
# Large sample (150 samples)
large_sample = feature

# MLE Mean
mle_mean_large = np.mean(large_sample)

# MLE Variance
mle_variance_large = np.var(large_sample)

# Bayesian Estimation
bayesian_mean_large = (
    prior_weight * prior_mean +
    len(large_sample) * mle_mean_large
) / (prior_weight + len(large_sample))

print("Large Sample (150 Samples)")
print("MLE Mean:", mle_mean_large)
print("MLE Variance:", mle_variance_large)
print("Bayesian Mean:", bayesian_mean_large)
```

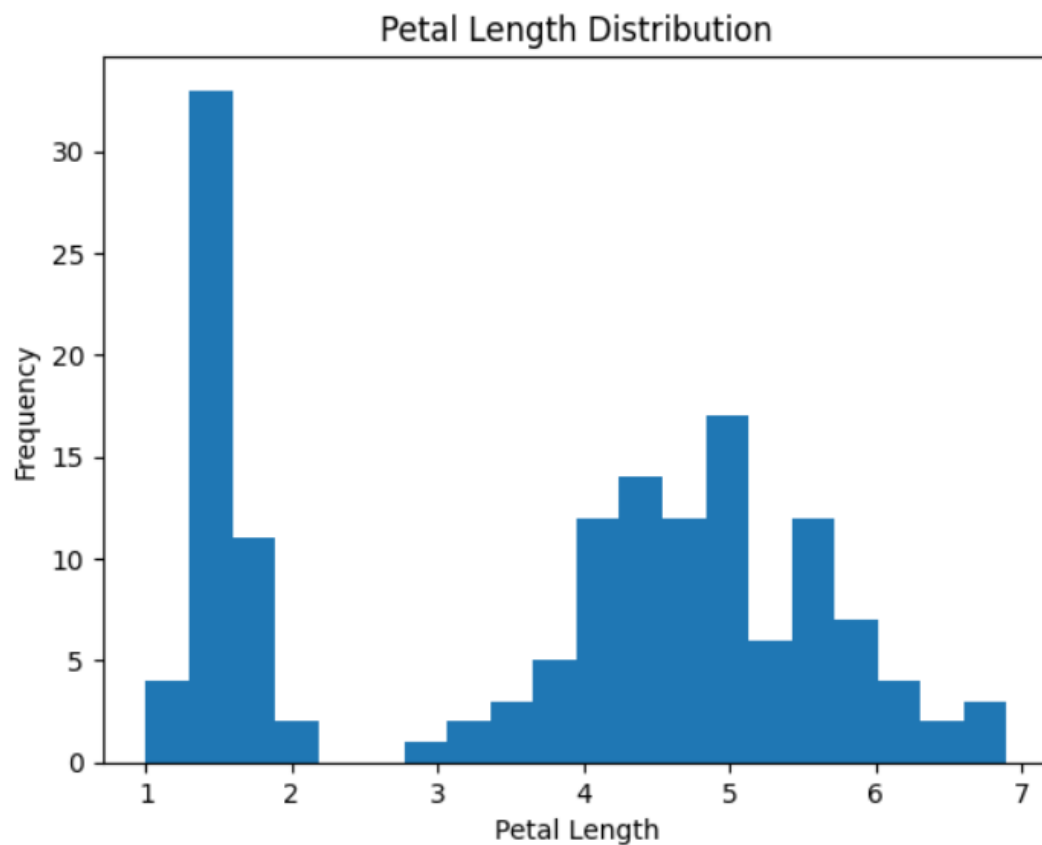
```
Large Sample (150 Samples)
MLE Mean: 3.7580000000000005
MLE Variance: 3.0955026666666665
Bayesian Mean: 3.7731250000000003
```

Histogram

```
plt.hist(feature, bins=20)

plt.title("Petal Length Distribution")
plt.xlabel("Petal Length")
plt.ylabel("Frequency")

plt.show()
```



Comparison Table

	Sample Size	MLE Mean	MLE Variance	Bayesian Mean
0	20 Samples	1.435	0.020275	2.290000
1	150 Samples	3.758	3.095503	3.773125